# Closest Pair Problem

**Lydia Ballard, Sydney Frisbee**

# What is the Closest Pair Problem?

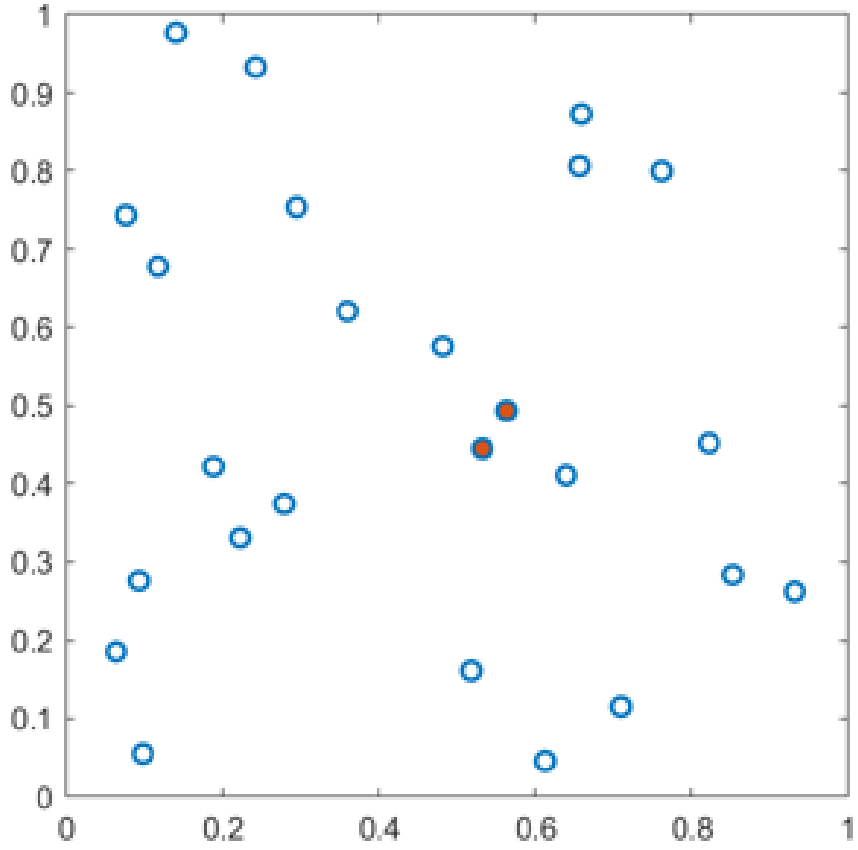
Closest Pair Problem – a problem that finds the two closest points in a metric space, typically using Euclidean distance, for a set of n points.

We plan to solve this problem with the following algorithms:

- Brute Force
- Divide and Conquer
- Line Sweep
- Bee Colony Optimization (BCO)

# Objectives

- How does runtime grow for each algorithm as input size $n$ increases
- At what input size does Divide & Conquer outperform Brute Force
- Which algorithm is best for solving the closest pair problem



# Brute Force

Brute force figures out the distance between every pair and stores the minimum distance found.

Time Complexity: $\Theta(n^2)$

We have this algorithm as a baseline due to it being very straightforward

# Divide and Conquer

Points are recursively divided into smaller groups. Minimum distance is calculated within each group. During the merging step, check for possible closer pairs across the dividing point.

- Time complexity: $\Theta$ (n log n)
- Reduces the time complexity compared to just brute force, so it is better for more data
- More complex to understand and implement

# Line Sweep

Line Sweep algorithm first sorts all points by x-coordinate and then processes them from left to right using a sweep line.

Time Complexity: $\Theta(n \log n)$

We have this algorithm because it is a specialized algorithm and is useful for comparisons with general algorithms

# Bee Colony Optimization

Employed Bees: takes the current pair and compares it to a neighboring pair, keeping the closer one

Onlooker Bees: computes fitness for pairs and randomly picks a promising pair to attempt to find a closer distance

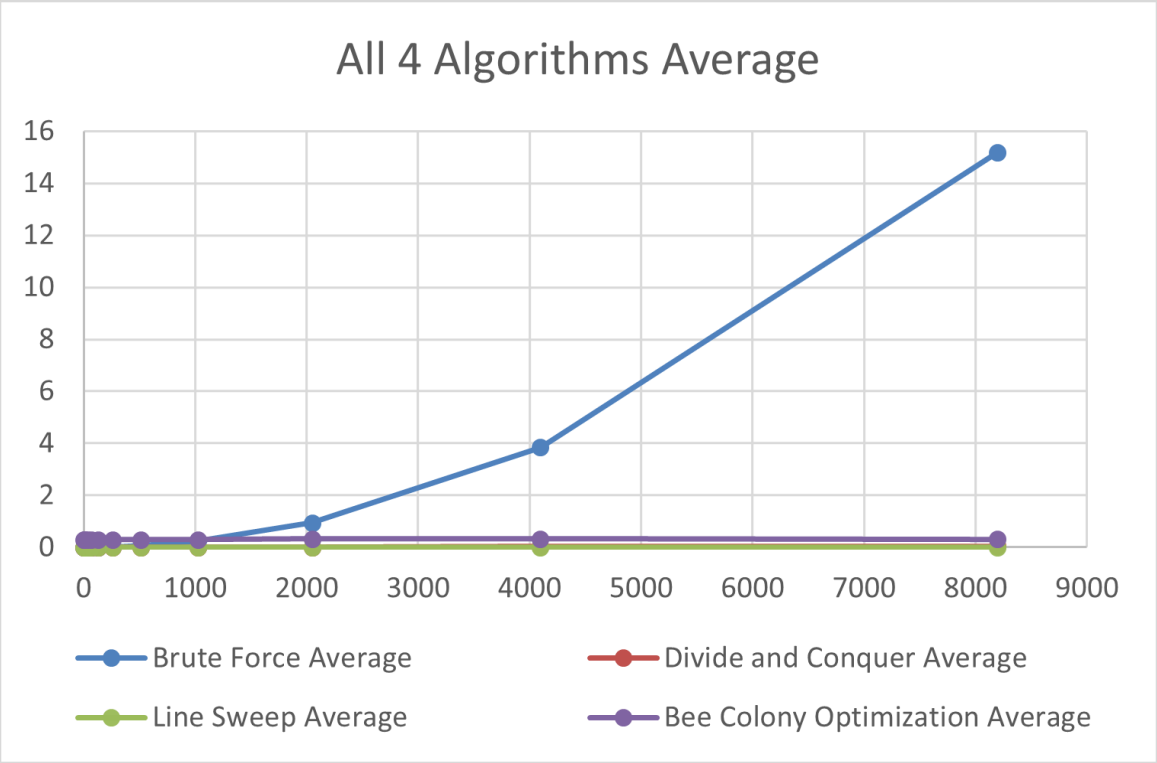
Scout Bees: determines that if the pair hasn't improved after a specific number of iterations, then it may be a bad candidate and replaces the pair with a random pair

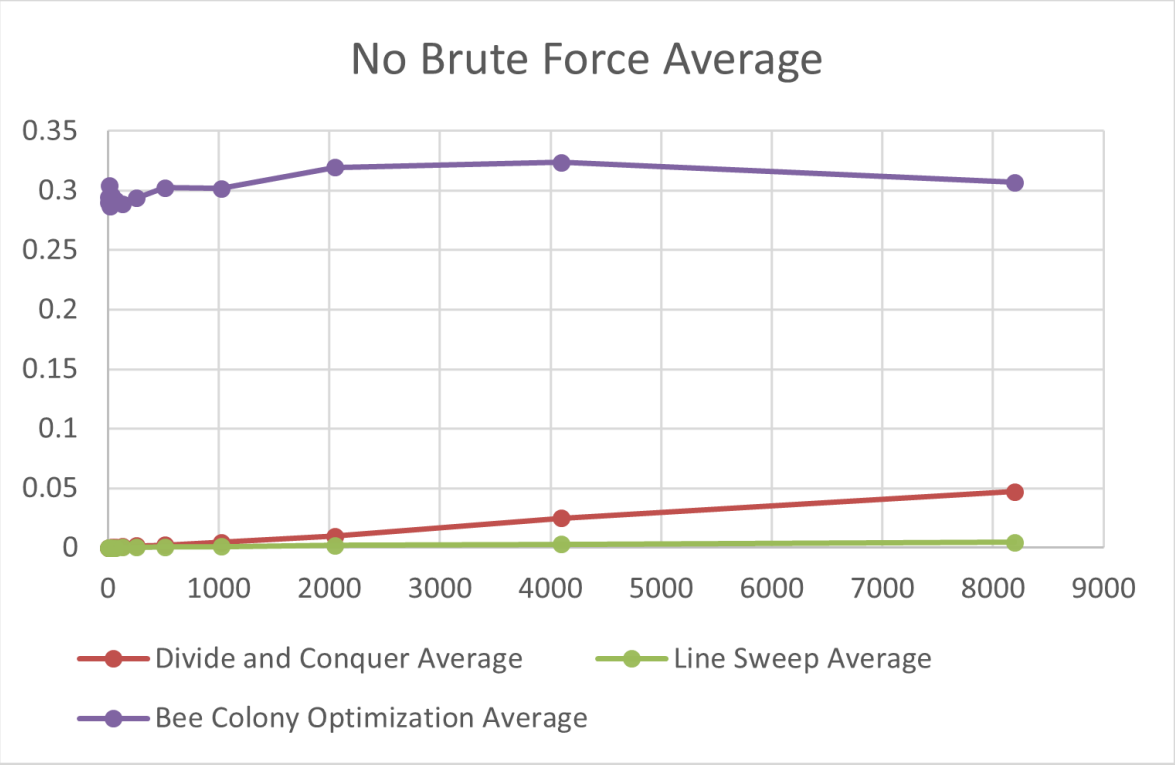
- Time Complexity: $\Theta(N \cdot I)$
- Heuristic

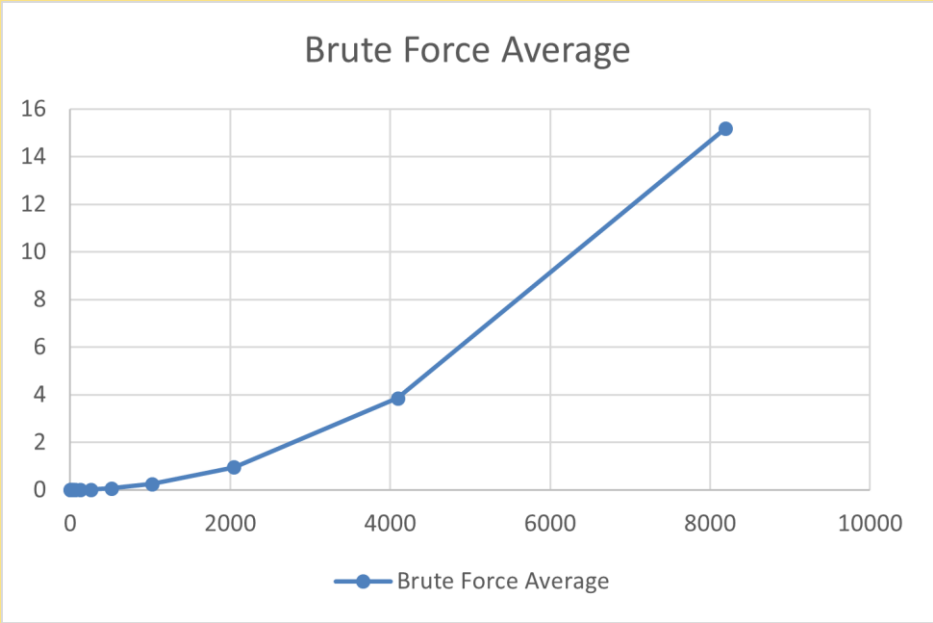
# Experiment Description

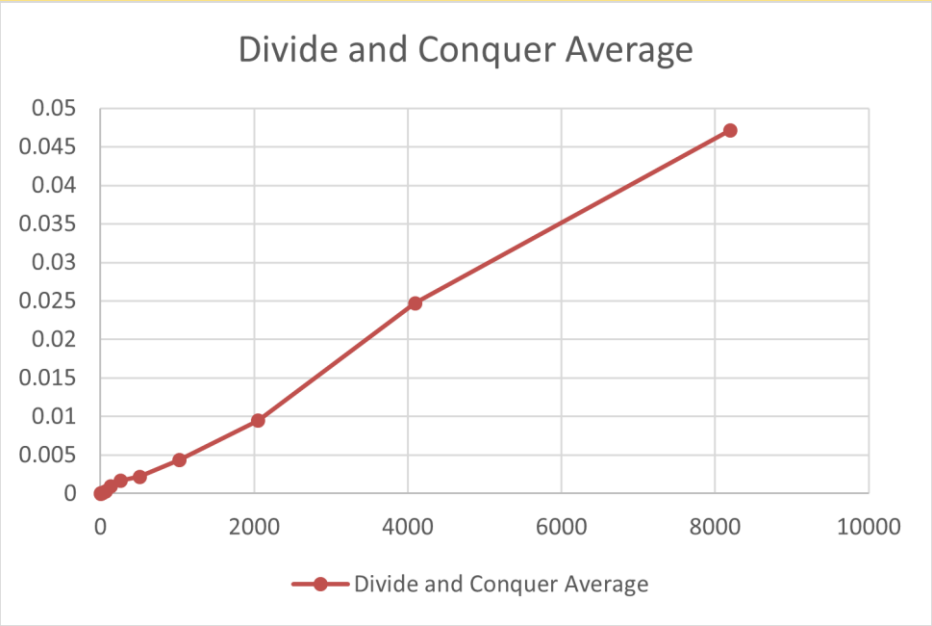
- Coded in Python
- Artificially generated datasets of XY pairs with a size n
- n starts at 2 and increases exponentially up to 8192
- Runs each algorithm 5 times for each size n
- Specifications of Test Computer:
  - OS: Windows 11
  - CPU: AMD Ryzen 9 6900HS Creator Edition
  - GPU: Nvidia GeForce 3050 Laptop
  - RAM: 32 GB

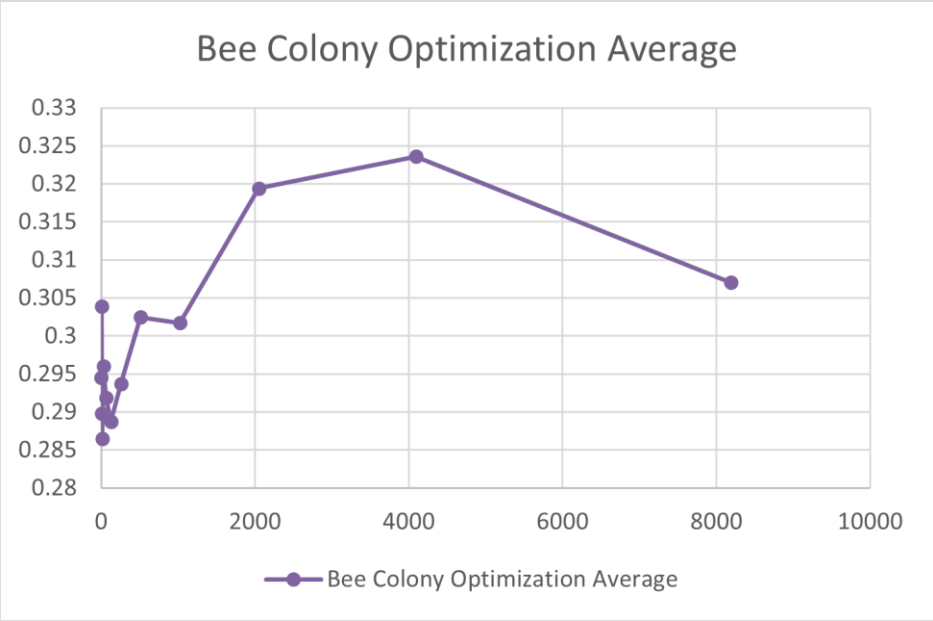
# Results

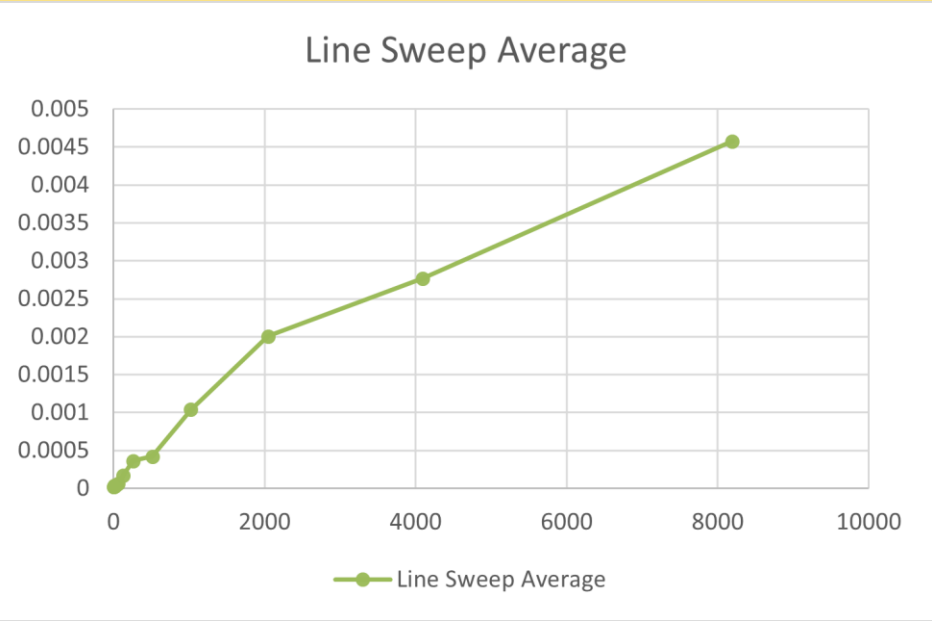

All 4 Algorithms Average
16
14
12
10
8
6
4
2
0
0
1000
2000
3000
4000
5000
6000
7000
8000
9000
Brute Force Average
Divide and Conquer Average
Line Sweep Average
Bee Colony Optimization Average


No Brute Force Average
0.35
0.3
0.25
0.2
0.15
0.1
0.05
0
0
1000
2000
3000
4000
5000
6000
7000
8000
9000
Divide and Conquer Average
Line Sweep Average
Bee Colony Optimization Average


Brute Force Average
16
14
12
10
8
6
4
2
0
0
2000
4000
6000
8000
10000
Brute Force Average


Divide and Conquer Average
0.05
0.045
0.04
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0
0
2000
4000
6000
8000
10000
Divide and Conquer Average


Bee Colony Optimization Average
0.33
0.325
0.32
0.315
0.31
0.305
0.3
0.295
0.29
0.285
0.28
0
2000
4000
6000
8000
10000
Bee Colony Optimization Average


Line Sweep Average
0.005
0.0045
0.004
0.0035
0.003
0.0025
0.002
0.0015
0.001
0.0005
0
0
2000
4000
6000
8000
10000
Line Sweep Average

# Interpretations

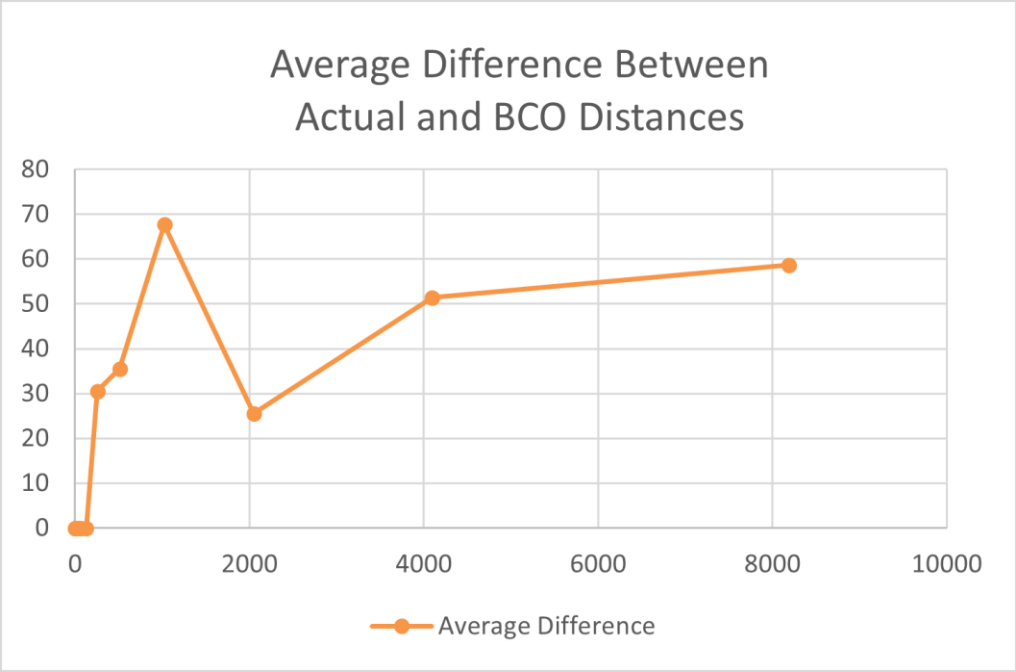
Brute Force is the slowest with $\Theta(n^2)$, while Divide and Conquer and Line Sweep are faster with $\Theta(n \log n)$. BCO is fast but less reliable since it does not always give exact results.

Divide and Conquer becomes much faster than Brute Force as input size increases, showing the impact of better time complexity. Divide and Conquer specifically gets better at $n = 16$.

With our data, we can conclude that Line Sweep is the best algorithm to solve the Closest Pair problem.

# Reflection

- Deciding to generate datasets
- Seeing the differences between the result BCO got and what the actual closest pair was, and how that changed (n = 256)
- Comparing our algorithm results with 3D Closest Pair Algorithms



# Questions

# Closest Pair Problem

**Lydia Ballard, Sydney Frisbee**